Phytium飞腾

# OpenCV 编译手册

**(V1.1)**

2023 年 2 月

飞腾信息技术有限公司

www.phytium.com.cn

## 版权声明：

本文档用于指导用户的相关应用和开发工作，版权归属飞腾信息技术有限公司所有，受法律保护。任何未经书面许可的公开、复制、转载、篡改行为将被依法追究法律责任。

## 免责声明：

本文档仅提供阶段性数据，并不保证该等数据的准确性及完整性。飞腾信息技术有限公司对此文档内容享有最终解释权，且保留随时更新、补充和修订的权利。所有资料如有更改，恕不另行通知。

如有技术问题，可联系 support@phytium.com.cn 获取支持，因不当使用本文档造成的损失，本公司概不承担任何责任。

Phytium 飞腾

# 当前版本

| 文件标识 | |
|---|---|
| 当前版本 | **1.1** |
| 完成日期 | 2023.02 |

# 版本历史

| 版本 | 修订时间 | 修订人 | 修订内容 |
|---|---|---|---|
| V1.1 | 2023.02 | 刘才诚 | 初稿 |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

Phytium 飞腾

# 目 录

1 简介 ..... 1
2 安装 Opencv ..... 3
2.1 硬件环境 ..... 3
2.2 系统环境 ..... 3
2.3 安装依赖 ..... 3
2.4 获取 OpenCV 源代码 ..... 4
2.5 使用 CMake 从源代码构建 OpenCV ..... 5
3 更改配置 ..... 7
4 卸载 opencv ..... 8
5 交叉编译 Opencv ..... 9
5.1 编译依赖 ..... 9
5.1.1 编译 libz ..... 9
5.1.2 编译 yasm ..... 10
5.1.3 编译 xvidcore ..... 10
5.1.4 编译 libpng ..... 11
5.1.5 编译 x264 ..... 12
5.1.6 编译 jpegsrc ..... 12
5.1.7 编译 ffmpeg ..... 13
5.2 编译 OpenCV ..... 13
5.3 移植 opencv ..... 15

Phytium飞腾

# 1 简介

本文档主要介绍了基于 D2000+X100 的 OpenCV 编译方式，包括源码编译、交叉编译、安装和卸载，旨为在飞腾 D2000+X100 编译安装 OpenCV 的图像处理和计算机视觉方面的开发者提供参考。

OpenCV 是一个基于 BSD 许可（开源）发行的跨平台计算机视觉和机器学习软件库，可以运行在 Linux、Windows、Android 和 MacOS 操作系统上。它轻量级而且高效——由一系列 C 函数和少量 C++类构成，同时提供 Python、Ruby、MATLAB 等语言的接口，实现了图像处理和计算机视觉方面的很多通用算法。

OpenCV 用 C++语言编写，它具有 C++，Python，Java 和 MATLAB 接口，并支持 Windows，Linux，Android 和 Mac OS，OpenCV 主要倾向于实时视觉应用，并在可用时利用 MMX 和 SSE 指令， 如今也提供对于 C#、Ch、Ruby，GO 的支持。OpenCV 的应用领域非常广泛，包括图像拼接、图像降噪、产品质检、人机交互、人脸识别、动作识别、动作跟踪、无人驾驶等。OpenCV 还提供了机器学习模块，可以使用正态贝叶斯、K 最近邻、支持向量机、决策树、随机森林、人工神经网络等机器学习算法。

- OpenCV 应用层语言：Python、Java、C；
- OpenCV 操作系统：Windows、Linux、MacOSx、iOS、Android、WinRTx；
- OpenCV 支持的硬件：GPU、x86、x64、ARM、MIPS；
- OpenCV 中线程：
  1） Concurrency：Windows；
  2） GCD：MacOSx、IOS；
  3） TBB：Windows、MacOSx、Linux、Android、（WinRT)。
- OpenCV 编程中使用的加速手段：
  1) 针对 GPU 的加速：CUDA、OpenCL；
  2) 针对 x86 和 x64 加速：OpenCL、SSE/AUX；
  3) 针对 ARM 加速：NEON。

虽然 OpenCV 的目标是能够实时的处理计算机视觉问题，但是大多数计算机视觉的算法太过复杂，在 CPU 上执行效率很低。而计算机视觉的大多数算法具有天生的并行性，非常适合在 GPU 和 NPU 上执行并获得可观的加速比。

Phytium飞腾

飞腾套片 X100 是一款微处理器的配套芯片，主要功能包括图形图像处理和接口扩展两类。目前飞腾 X100 套片已经支持 OpenCL 3.0 标准，这为 OpenCV 算法在 X100 进行加速计算提供了中间媒介。在安装完 OpenCV 后，就可以调用其中的 OpenCL 模块，将 OpenCV 算法无缝迁移到 OpenCL 上，OpenCL 把在 CPU 上执行的算法转换成 kernel 函数移植到 X100 上执行，从而利用强大的 GPU 算力为 OpenCV 算法提供数倍的加速。

Phytium飞腾

# 2 安装 Opencv

OpenCV 的安装和测试使用 D2000+X100 套片开发板如图 2-1。

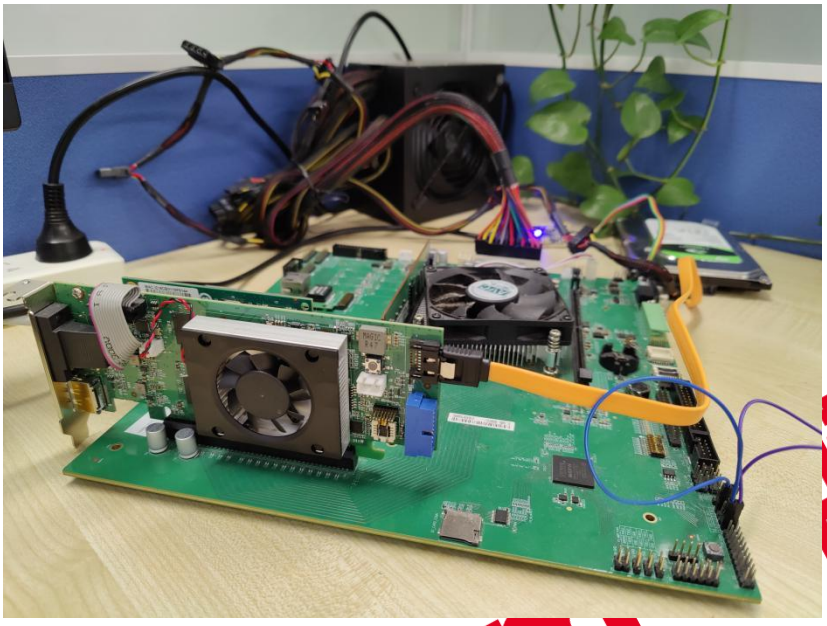

图 2-1 X100 套片开发环境

## 2.1 硬件环境

OpenCV 安装和测试的硬件环境包括下表 2-1 中的硬件设备。

表 2-1 硬件规格

| 项目 | 说明 |
|---|---|
| CPU | FT D2000 |
| GPU | X100 套片 |
| 网络 | 可访问外网 |
| 内存 | DDR4 8G |
| 存储 | 机械硬盘 1TB |

## 2.2 系统环境

本次适配使用如下操作系统。

表 2-2 操作系统规格

| 项目 | 说明 | 终端查询命令 |
|---|---|---|
| Ubantu | 20.04.5 LTS | lsb_release -a |
| Kernel | 4.19.115 + aarch64 | uname -ar |
| OpenCL | 3.0 | clinfo |

## 2.3 安装依赖

在终端下执行以下命令安装相关依赖库。

安装通用工具：

Phytium飞腾

```
sudo apt install build-essential cmake pkg-config unzip yasm git checkinstall
```

安装图像 I/O 库：

```
sudo apt install libjpeg-dev libpng-dev libtiff-dev
```

安装视频/音频库- FFMPEG、GSTREAMER、x264 等:

```
sudo apt install libavcodec-dev libavformat-dev libswscale-dev libavresample-dev
sudo apt install libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev
sudo apt install libxvidcore-dev x264 libx264-dev libfaac-dev libmp3lame-dev libtheora-dev
sudo apt install libfaac-dev libmp3lame-dev libvorbis-dev
```

安装相机编程接口库:

```
sudo apt install libdc1394-22 libdc1394-22-dev libxine2-dev libv4l-dev v4l-utils
```

安装来自 OpenCV highghui 模块的图形用户功能的 GTK 库：

```
sudo apt install libgtk-3-dev
```

安装用于 CPU 的并行库 C++:

```
sudo apt install libtbb-dev
```

安装 OpenCV 的优化库:

```
sudo apt install libatlas-base-dev gfortran
```

## 2.4 获取 OpenCV 源代码

下载 opencv 和 opencv_contrib，两种方式：从官网直接下载；从 Git Repository 获取。

- 官网下载：https://opencv.org/releases/ 。进入官网可选择不同发行的 OpenCV 版本，Linux 环境下点击 Sources 即可下载 OpenCV 源代码。

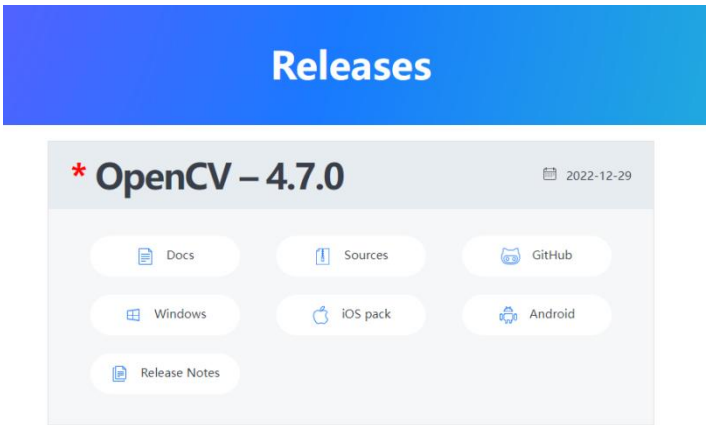

图 2-2 OpenCV 官网

右键提取到此处即可，或者在终端输入命令：

```
unzip opencv-4.7.0.zip
```

- 从 Git Repositoy 获取最新的 OpenCV 源代码：

启动 Git 客户端并克隆 OpenCV 存储库。如果您需要 OpenCV contrib 存储库中的模块，那么也可以克隆它。

```
mkdir opencv_work/
cd opencv_work/
git clone https://github.com/opencv/opencv.git
git clone https://github.com/opencv/opencv_contrib.git
```

如果需要切换到 opencv 历史的发行版本，即可在终端执行：

```
cd opencv/
git check xxx
cd ../opencv_contrib
git check xxx
```

注：xxx 应为对应的版本号例如 3.4.1，并且扩展库 opencv_contrib 应与 opencv 的版本号一一对应。

## 2.5 使用 CMake 从源代码构建 OpenCV

创建一个临时目录，我们将其指定为<cmake_build_dir>，您要将生成的 Makefile，项目文件以及目标文件和输出二进制文件放在那里。

```
cd opencv/
mkdir build
cd build
```

配置:运行 cmake [<some optional parameters>] <OpenCV 源目录的路径>

```
cmake \
-D CMAKE_INSTALL_PREFIX=/usr/local \
-D CMAKE_BUILD_TYPE=RELEASE \
-D WITH_OPENGL=ON \
-D WITH_TBB=ON \
```

```
-D WITH_GTK=ON \
-D WITH_FFMPEG=ON \
-D INSTALL_C_EXAMPLES=ON \
-D OPENCV_GENERATE_PKGCONFIG=ON \
-D OPENCV_EXTRA_MODULES_PATH=/home/lcc/opencv_work/
opencv_contrib/modules\
..
```

表 2-3 参数配置说明

| 参数选项 | 说明 |
|---|---|
| CMAKE_INSTALL_PREFIX | 动态库安装路径，可自定义 |
| CMAKE_BUILD_TYPE | 发布版本 |
| WITH_OPENGL | OpenGL 集成可用于绘制具有以下后端的硬件加速窗口：GTK、WIN32 和 Qt |
| WITH_TBB | 开放 TBB，Threading Building Blocks 是一个用于并行编程的跨平台库。 |
| WITH_GTK | GTK GUI 库来绘制窗口 |
| WITH_FFMPEG | 用于解码和编码视频文件和网络流 |
| INSTALL_C_EXAMPLES | 从 samples/cpp 目录安装 C++示例源 |
| OPENCV_GENERATE_PKGCONFIG | 生成启用.pc 文件以及标准 CMake 包 |
| OPENCV_EXTRA_MODULES_PATH | 添加额外模块 |

编译：从构建目录执行 make，建议在多个线程中执行此操作。

```
sudo make - j$(nproc)
```

安装库，请从构建目录执行以下命令：

```
sudo make install
```

Phytium飞腾

# 3 更改配置

修改 opencv.conf 文件，在此空文件中添加 opencv 库的安装路径：/usr/local/lib。

```
sudo gedit /etc/ld.so.conf.d/opencv.conf
```

更新系统共享链接库：

```
sudo ldconfig
```

配置 bash ，修改 bash.bashrc 文件：

```
sudo gedit /etc/bash.bashrc
```

在文件末尾加入：

```
PKG_CONFIG_PATH=$PKG_CONFIG_PATH:/usr/local/lib/pkgconfig
export PKG_CONFIG_PATH
```

保存退出，然后执行如下命令使得配置生效：

```
source /etc/bash.bashrc
```

最后，查看 opencv 的版本信息，配置成功。

```
pkg-config --modversion opencv4
```

Phytium 飞腾

# 4 卸载 opencv

进入 opencv 的编译目录，执行以下命令：

```
cd build
sudo make uninstall
cd ..
sudo rm -rf build/
```

删除安装在 lib 和 include 的 opencv：

```
sudo rm -r /usr/local/include/opencv2 /usr/local/include/opencv /usr/include/opencv /usr/include/opencv2 /usr/local/share/opencv /usr/local/share/OpenCV /usr/share/opencv /usr/share/OpenCV /usr/local/bin/opencv* /usr/local/lib/libopencv*
```

检查是否删除完：

```
pkg-config opencv4 --libs
pkg-config opencv4 --cflags
pkg-config opencv4 --modversion
```

彻底清除相关包：

```
sudo apt-get autoremove opencv-doc opencv-data libopencv-dev libopencv2.4-java libopencv2.4-jni python-opencv libopencv-core2.4 libopencv-gpu2.4 libopencv-ts2.4 libopencv-photo2.4 libopencv-contrib2.4 libopencv-imgproc2.4 libopencv-superres2.4 libopencv-stitching2.4 libopencv-ocl2.4 libopencv-legacy2.4 libopencv-ml2.4 libopencv-video2.4 libopencv-videostab2.4 libopencv-objdetect2.4 libopencv-calib3d2.4
```

# 5 交叉编译 Opencv

除了在 D2000 下编译 OpenCV，还可以在服务器上部署交叉编译环境，提高 OpenCV 的编译效率。本次交叉编译环境如下:

表 5.1 交叉编译信息

| 选项 | 说明 |
|---|---|
| 编译平台 | Ubuntu20.04 x86 |
| 目标平台 | D2000 ubuntu20.04 phytium-linux-buildroot ARM |
| 交叉编译工具 | aarch64-linux-gnu-gcc |
| 编译方式 | 使用 Cmake 进行交叉编译 |

## 5.1 编译依赖

首先需要准备好 opencv 运行依赖的库文件（libz，yasm，xvidcore，libpng，x264，jpegsrc，ffmpeg），创建个文件夹放进去。

配置环境变量：将下列命令行的环境变量 OPENCV_3THPARTY_LIB 添加到 /etc/bash.bashrc 文件结尾处。

```
vim /etc/bash.bashrc
export OPENCV_3THPARTY_LIB=/home/lcc/opencv-work/opencv_3p_lib
source /etc/bash.bashrc
```

在 shell 中就可以用$OPENCV_3THPARTY_LIB 代替后面的目录，后面的目录是用户自己指定的，用来存放依赖库编译的输出。

使用以下命令检查你的环境变量是否设置成功：

```
echo $OPENCV_3THPARTY_LIB
```

对 OpenCV 依赖库进行解压缩、编译并安装。注意这些库文件之间还有相互的依赖性，具体依赖性在官网有介绍，一定要按照下面的顺序安装，否则会报错。

### 5.1.1 编译 libz

对 libz 进行解压缩：

```
tar zxvf zlib-1.2.11.tar.gz
```

进入解压后的文件夹，创建一个新的 build 文件夹，进入 build 文件夹，创建一个脚本文件 build.sh。

```
cd zlib-1.2.11/
mkdir build
cd build
```

```
vim build.sh
```

在脚本中输入以下命令：

```
CC=aarch64-linux-gnu-gcc ../configure --prefix=$OPENCV_3THPARTY_LIB
--shared
make
make install
```

运行脚本进行编译安装，在安装目录下就有了输出文件。

```
sudo chmod 777 build.sh
./build.sh
```

### 5.1.2 编译 yasm

对 yasm 进行解压缩：

```
tar zxvf yasm-1.3.0.tar.gz
```

进入解压后的文件夹，创建一个新的 build 文件夹，进入 build 文件夹，创建一个脚本文件 build.sh。

```
cd yasm-1.3.0/
mkdir build
cd build
vim build.sh
```

在脚本中输入以下命令：

```
../configure --host=aarch64-linux-gnu --prefix=$OPENCV_3THPARTY_LIB
--enable-shared --enable-static
make
make install
```

配置权限并运行：

```
sudo chmod 777 build.sh
./build.sh
```

### 5.1.3 编译 xvidcore

解压：

```
tar zxvf xvidcore-1.3.7.tar.gz
```

进入解压的文件夹，已经有 build 文件夹了，进去找 makefile，它在 build/generic 路径下。

```
cd build/generic/
vim build.sh
```

输入以下内容：

```
../configure --host=aarch64-linux-gnu --prefix=$OPENCV_3THPARTY_LIB
--disable-assembly
make
sudo make install
```

这里重新生成了以下 Makefile，注意最后安装需要加 sudo 获取权限。

### 5.1.4 编译 libpng

先解压：

```
tar zxvf libpng-1.6.37.tar.gz
```

libpng 不需要再自己 configure 出 makefile 文件了，在 scripts 文件夹下已经有 makefile.linux 文件，直接修改它就行，如果自己 configure 可能会遇到找不到 zlib 的问题。先把它拷贝到当前路径下，重命名为 Makefile，然后打开编辑它。

```
cd libpng-1.6.37/
cp scripts/makefile.linux Makefile
vim Makefile
```

按住 crtl+f 搜索 zlib，找到下面的语句，修改成用户自己的路径，本次编译直接用环境变量。修改前：

```
# Where the zlib library and include files are located.
#ZLIBLIB=/usr/local/lib
#ZLIBINC=/usr/local/include
ZLIBLIB=../zlib
ZLIBINC=../zlib
```

修改后：

```
# Where the zlib library and include files are located.
ZLIBLIB=$(OPENCV_3THPARTY_LIB)lib
ZLIBINC=$(OPENCV_3THPARTY_LIB)include
#ZLIBLIB=../zlib
#ZLIBINC=../zlib
```

再找到 prefix，把它修改成你想输出的路径，修改前：

```
# Prefix must be a full pathname.
prefix=/usr/local
exec_prefix=$(prefix)
```

修改后：

```
# Prefix must be a full pathname.
prefix=$(OPENCV_3THPARTY_LIB)
exec_prefix=$(prefix)
```

再找到 CC，修改为自己的交叉编译器 aarch64-linux-gnu-gcc 即可。

在 shell 中输入编译安装命令：

```
make
make install
```

### 5.1.5 编译 x264

解压，创建编译脚本：

```
tar xvf x264.tar.gz
cd x264/
mkdir build
cd build
vim build.sh
```

在 build 脚本中添加以下命令：

```
CC=aarch64-linux-gnu-gcc ../configure --host=aarch64-linux-gnu
--prefix=$OPENCV_3THPARTY_LIB --enable-shared --disable-asm
make
sudo make install
```

保存后，运行脚本。

### 5.1.6 编译 jpegsrc

和之前同理，本次脚本内容为：

```
../configure --prefix=$OPENCV_3THPARTY_LIB CC=aarch64-linux-gnu-gcc
--host=aarch64-linux-gnu
make
make install
```

### 5.1.7 编译 ffmpeg

ffmpeg 编译脚本如下：

```
../configure --prefix=$OPENCV_3THPARTY_LIB --enable-shared --disable-static
--enable-gpl --enable-cross-compile --arch=aarch64 --disable-stripping
--target-os=linux --enable-libx264 --enable-libxvid --cc=aarch64-linux-gnu-gcc
--enable-swscale --extra-ldflags=-L$OPENCV_3THPARTY_LIB/lib
--extra-cflags=-I$OPENCV_3THPARTY_LIB/include
make
make install
```

编译安装完成后，进入安装目录的 lib，依赖库都存放在这个文件夹。

## 5.2 编译 OpenCV

OpenCV 的源码下载如 2.4 小节所示。

在编译 OpenCV 之前一定先设置环境变量，否则会找不到 ffmpeg 等错误。将 pkg-config（ffmpeg...）添加到~/.bashrc 文件中。

```
export PKG_CONFIG_PATH=$OPENCV_3THPARTY_LIB/lib/pkgconfig
export LD_LIBRARY_PATH=$OPENCV_3THPARTY_LIB/lib
export PKG_CONFIG_LIBDIR=$PKG_CONFIG_LIBDIR:
$OPENCV_3THPARTY_LIB/lib
```

打开 opencv 文件夹中的 aarch64-gnu.toolchain 文件，修改 cmake。

```
vim /home/lcc/opencv_work/opencv/platforms/linux/aarch64-gnu.toolchain.cmake
```

修改的内容如下：

```
set( CMAKE_SYSTEM_NAME Linux )
set( CMAKE_SYSTEM_PROCESSOR aarch64)
set(GCC_COMPILER_VERSION "" CACHE STRING "GCC Compiler version")
set(GNU_MACHINE "aarch64-linux-gnu" CACHE STRING "GNU compiler triple")
set( OPENCV_ENABLE_PKG_CONFIG ON)
set( CMAKE_C_COMPILER aarch64-linux-gnu-gcc )
set( CMAKE_CXX_COMPILER aarch64-linux-gnu-g++ )
set( CMAKE_C_FLAGS "-Wl,-rpath-link=$OPENCV_3THPARTY_LIB/lib")
include("${CMAKE_CURRENT_LIST_DIR}/arm.toolchain.cmake")
```

进入到 opencv/build 目录下生成 makefile：

```
cmake \
-D CMAKE_INSTALL_PREFIX=/home/lcc/opencv_work/opencv_install \
-D CMAKE_BUILD_TYPE=RELEASE \
-D CMAKE_TOOLCHAIN_FILE=../platforms/linux/aarch64-gnu.toolchain.cmake
-D WITH_FFMPEG=ON \
-D OPENCV_GENERATE_PKGCONFIG=ON \
-D OPENCV_EXTRA_MODULES_PATH = /home/lcc/opencv_work/
opencv_contrib/modules \
..
```

检查 ffmpeg，以及之前编译的库是否能找到。没问题就和下图一样，开始编译和安装。

```
  Media I/O:
    ZLib:                        build (ver 1.2.11)
    JPEG:                        libjpeg-turbo (ver 2.0.2-62)
    WEBP:                        build (ver encoder: 0x020e)
    PNG:                         build (ver 1.6.37)
    TIFF:                        build (ver 42 - 4.0.10)
    JPEG 2000:                   build (ver 1.900.1)
    HDR:                         YES
    SUNRASTER:                   YES
    PXM:                         YES
    PFM:                         YES

  Video I/O:
    DC1394:                      NO
    FFMPEG:                      NO
      avcodec:                   YES (58.54.100)
      avformat:                  YES (58.29.100)
      avutil:                    YES (56.31.100)
      swscale:                   YES (5.5.100)
      avresample:                NO
    GStreamer:                   NO
    v4l/v4l2:                    YES (linux/videodev2.h)
```

图 5-1 Cmake 配置中 ffmpeg 信息

```
sudo make -j8
sudo make install
```

## 5.3 移植 opencv

编译安装完成，将 install 目录的所有文件移植到开发板中，设置 D2000 的环境变量 export LD_LIBRAR_PATH 为放置的 opencv libs 的路径 ，配置细节参考第 3 章。